numaring (NumaRing)

A Cache-Conscious, Topology-Aware MPMC Queue for High-Core NUMA Architectures

Theoretical Foundations & Research Specification Document | Domain: Microarchitecture & Concurrent Systems

---

> **[THEORETICAL ARCHITECTURE OVERVIEW]**
> numaring is a high-performance Multi-Producer Multi-Consumer (MPMC) concurrent queue architecture engineered conceptually to resolve cross-socket cache-line bouncing (MESI/MOESI invalidation loops) across multi-socket NUMA systems. By structuring memory hierarchy around node-local ring buffers, batched cross-node transfers, 128-byte cache-line padding, and fine-grained memory barriers, numaring establishes a framework for near-linear throughput scaling on high-core server CPUs.

1. Research Motivation & The Hardware Bottleneck

Modern high-core server CPUs (such as AMD EPYC, Intel Xeon, and Ampere ARM processors) feature multi-socket and multi-chip-module (MCM) topologies structured into distinct **Non-Uniform Memory Access (NUMA) domains**. Memory access latencies and interconnect bandwidth (Intel UPI or AMD Infinity Fabric) vary significantly depending on whether a CPU core accesses memory controller channels local to its socket or remote node memory.

Traditional state-of-the-art lock-free MPMC queues rely on a single global ring buffer with centralized atomic head and tail pointers updated via Compare-And-Swap (CAS) instructions. Under high thread contention across NUMA sockets, this centralized design introduces severe physical microarchitectural degradation:

- **Interconnect Cache-Line Bouncing:** When a CPU core on Socket 0 mutates a central atomic variable, the MESI/MOESI protocol invalidates the target cache line across all cores on Socket 1. Socket 1 cores experience hardware stalls of 100–300 CPU cycles while fetching the updated cache line over the interconnect.
- **Spatial Prefetcher False Sharing:** Modern CPU L2 spatial prefetchers pull memory into caches in 128-byte dual-cache-line pairs. Standard structures aligned only to 64 bytes suffer from false sharing when adjacent atomic counters reside within the same 128-byte fetch window.
- **Pipeline Memory Barrier Stalls:** Over-reliance on sequential consistency memory ordering forces full system memory barriers, causing instruction execution pipelines to stall continuously while waiting for coherence acknowledgement across sockets.

2. numaring System Methodology & Architecture

numaring resolves interconnect hardware saturation by replacing global atomic ring buffers with a **hierarchical, topology-aware node-local queue architecture**. The theoretical methodology is organized around four architectural pillars:

2.1 Hierarchical NUMA-Local Sub-Queues

At system initialization, numaring inspects the hardware topology and instantiates independent sub-queues pinned directly to each NUMA node's physical memory controller. Memory pages backing each sub-queue are allocated explicitly from the local socket's node domain to ensure zero remote DRAM access during standard operations.

2.2 Topology-Aware Enqueue/Dequeue Routing

Producer and consumer threads dynamically query their current executing CPU core and route enqueue and dequeue requests directly to their socket-local sub-queue. This confines atomic state transitions, memory writes, and cache invalidation signals strictly within the socket's local L3 cache hierarchy.

2.3 Batched Cross-Node Work Stealing & Synchronization

To maintain global load balance without sacrificing local performance, numaring employs a **batched work-stealing mechanism**. When a node-local queue suffers from overflow or underflow, elements are transferred across NUMA nodes in contiguous block chunks (e.g., 16 or 32 items) in a single atomic operation. This reduces inter-socket coherence protocol traffic by up to **93.75%** compared to element-by-element synchronization.

2.4 Microarchitectural Cache Optimizations

To optimize instruction execution efficiency and cache locality, numaring applies three microarchitectural principles:

- **128-Byte Dual-Cache-Line Alignment:** All atomic tracking variables are padded to 128 bytes, insulating them from spatial prefetcher false sharing.
- **Explicit Software Memory Prefetching:** Data cell addresses are pre-fetched into L1 data caches ahead of consumer reads to hide memory access latency.
- **Acquire-Release Memory Ordering:** All atomic primitives rely on fine-grained acquire-release semantics rather than sequential consistency, allowing out-of-order execution engines to operate without pipeline flushes.

3. Expected Outputs & Benchmark Evaluation Targets

The evaluation phase of numaring aims to quantify performance improvements across four key microarchitectural and system-level metrics:

Evaluation Metric	Standard Global Lock-Free Baseline	numaring Expected Target

Throughput Scaling (32+ Concurrent Threads)	Collapses under heavy CAS contention (< 15M ops/sec)	Near-linear scaling across sockets (> 120M ops/sec)
Tail Latency (p99 & p99.9)	Spikes to microsecond scale (> 1,200 ns) due to interconnect stalls	Consistently sub-100 nanosecond tail latency
Cross-Socket Cache Bouncing (perf c2c)	High HITM (Hit In Modified Cache) state transfers across sockets	> 85% reduction in cross-socket HITM cache invalidations
NUMA Local Memory Ratio (numastat)	Random cross-node page allocations (~50% remote memory accesses)	> 98% local NUMA node memory hit ratio